

Plan GCP — de zéro au déploiement

Google Cloud n'est pas une compétence : c'est une carte. Une fois qu'on sait où se trouve chaque pièce et qui a le droit de la toucher, le reste n'est que de la tuyauterie. Ce document trace cette carte en entier, avec pour terrain d'application concret le pipeline déjà écrit pour `recutement-test`.

Pourquoi ce document : vous n'avez jamais déployé sur GCP. Chaque section part d'un mécanisme réel — pas d'un survol marketing — et se termine, quand c'est utile, par ce qu'il signifie concrètement pour `.github/workflows/ci-cd.yml` et le `README.md` déjà en place.

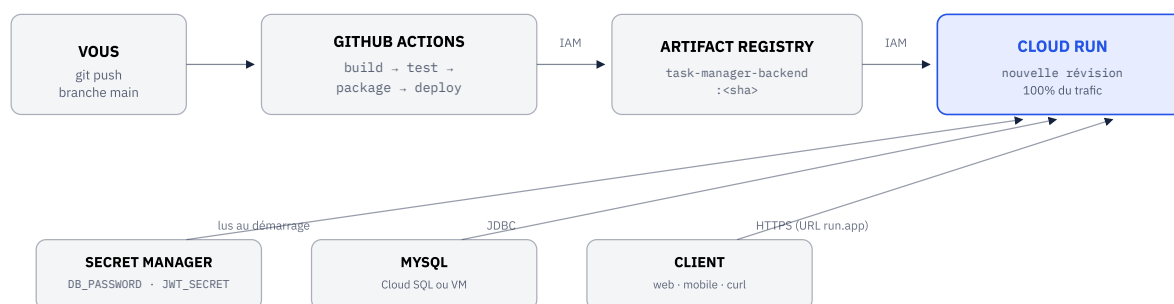
SOMMAIRE

- 01 · La grande image
- 02 · Hiérarchie des ressources
- 03 · IAM — qui peut faire quoi
- 04 · Authentifier GitHub Actions
- 05 · Activer des services
- 06 · Artifact Registry
- 07 · Cloud Run en détail
- 08 · Joindre MySQL
- 09 · Secret Manager
- 10 · Facturation
- 11 · Le pipeline, pas à pas
- 12 · Se lancer
- 13 · Lexique & pièges

La grande image

GCP est un catalogue d'une centaine de services loués à l'usage. Pour déployer ce backend, quatre d'entre eux suffisent : **Artifact Registry** (stocke l'image Docker), **Cloud Run** (l'exécute), **Secret Manager** (lui fournit ses secrets) et, en face, une base **MySQL** (Cloud SQL ou ailleurs). Un seul système transversal, **IAM**, décide qui a le droit de pousser une image ou de lire un secret — c'est le fil qui relie tout le reste de ce document.

Le schéma ci-dessous est la version condensée de tout ce qui suit : gardez-le en tête, chaque section détaille une de ses flèches.

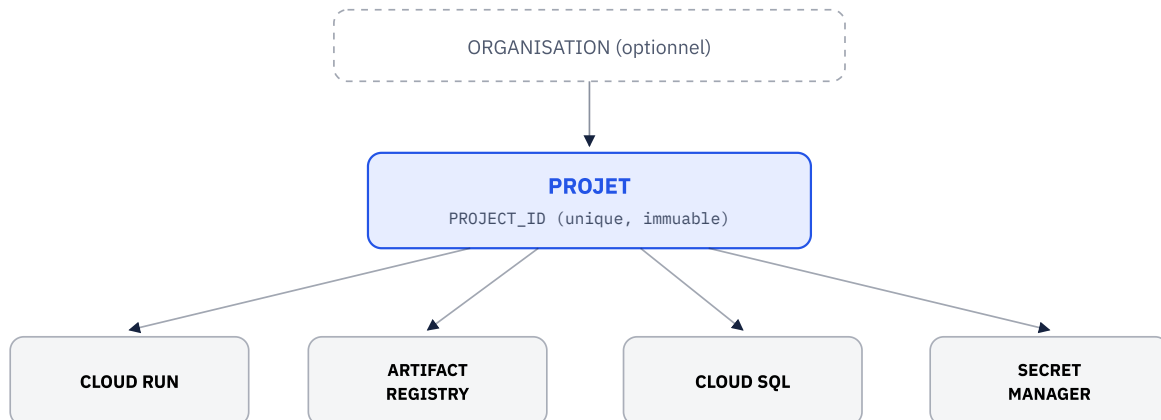


Un push sur `main` traverse quatre services GCP. Chaque flèche « IAM » est une vérification de permission qui peut échouer si le compte de service n'a pas le bon rôle — c'est le sujet des sections 03 et 04.

La hiérarchie des ressources

Rien n'existe dans GCP en dehors d'un **projet**. C'est l'unité de base : la facturation, les services activés, les permissions et les quotas sont tous rattachés à un projet précis. Une **Organisation** (liée à un domaine Google Workspace) peut regrouper plusieurs projets via des **dossiers**, mais avec un compte Google personnel — votre cas de départ — vous

créez simplement des projets directement, sans organisation ni dossier.
C'est parfaitement normal et suffisant.



Un rôle IAM accordé au niveau du projet s'applique par héritage à toutes les ressources qu'il contient — c'est exactement ce que fait le compte de service du pipeline (section 03).

Chaque projet porte **trois identifiants** qu'il ne faut pas confondre :

IDENTIFIANT	FORME	RÔLE
Nom du projet	« Task Manager »	Étiquette cosmétique, modifiable, non unique. N'apparaît dans aucune commande.
ID du projet	task-manager-472013	Unique <i>dans le monde entier</i> , immuable, en minuscules. C'est le <code>PROJECT_ID</code> utilisé dans chaque commande <code>gcloud</code> et dans le pipeline.
Numéro du projet	908741235690	Attribué automatiquement. Sert surtout en interne (ex. l'adresse du compte de service par défaut). Rarement tapé à la main.

03 — SÉCURITÉ

IAM — qui a le droit de faire quoi

IAM (Identity and Access Management) est le contrôle d'accès central de toute la plateforme : chaque appel d'API, humain ou automatisé, passe par une vérification IAM avant de s'exécuter. Une permission

accordée est toujours la combinaison de trois éléments — un **principal** (qui), un **rôle** (quoi) et une **ressource** (où) — appelée une *policy binding*.

- **Principal** — un compte Google humain, un **compte de service** (identité non humaine pour une application ou un pipeline — c'est ce qu'utilise GitHub Actions), un groupe, ou les principaux spéciaux `allUsers` / `allAuthenticatedUsers` (utilisés pour l'accès public : `-allow-unauthenticated` sur Cloud Run revient à accorder le rôle `run.invoker` à `allUsers`).
- **Rôle** — un paquet nommé de permissions fines. On n'accorde presque jamais une permission isolée (`run.services.create`) mais un rôle qui en regroupe plusieurs : *basique* (Éditeur/Propriétaire — très large, à éviter), *prédéfini* (scopé à un service précis, ex. `roles/run.admin` — ce qu'utilise le pipeline), ou *personnalisé* (assemblé à la main, usage avancé).
- **Ressource** — le projet entier, ou une ressource précise (un secret unique, un service Cloud Run unique).

Un **compte de service** est un compte « robot » avec sa propre adresse (`nom@PROJECT_ID.iam.gserviceaccount.com`), sans mot de passe : il s'authentifie soit avec une **clé JSON** téléchargée, soit sans aucun secret via la **Workload Identity Federation** — détaillé section suivante.

Les quatre rôles accordés dans ce projet

RÔLE	DONNE LE DROIT DE...
<code>roles/artifactregistry.writer</code>	pousser des images Docker dans les dépôts Artifact Registry du projet.
<code>roles/run.admin</code>	créer, mettre à jour et déployer des services Cloud Run.
<code>roles/iam.serviceAccountUser</code>	« agir en tant que » un autre compte de service — nécessaire pour déployer un service qui s'exécutera lui-même sous une identité de service (voir encadré ci-dessous).

`roles/secretmanager.secretAccessor`

lire la *valeur* d'un secret (distinct du droit de le créer ou de le gérer — c'est délibérément le rôle le plus étroit possible).

PIÈGE N°1

`iam.serviceAccountUser` se comprend rarement du premier coup. Un service Cloud Run ne s'exécute pas « dans le vide » : il tourne lui-même sous une identité de compte de service (par défaut, le *compte de service Compute par défaut* du projet). Déployer un service qui va « devenir » cette identité exige que le déployeur ait la permission d'« agir en tant que » elle. Sans ce rôle, le déploiement échoue avec une erreur du type `iam.serviceaccounts.actAs denied` — ce n'est pas un bug, c'est la protection contre l'élévation de privilèges déguisée en déploiement.

04 — IDENTITÉ DU PIPELINE**Authentifier GitHub Actions face à GCP**

GitHub Actions doit prouver son identité à GCP avant chaque appel. Deux mécanismes existent, avec un compromis très différent entre simplicité et surface de risque.



Même point de départ et d'arrivée ; ce qui change est l'étape du milieu. À gauche, un secret permanent dort dans les paramètres GitHub. À droite, rien n'est stocké : un jeton signé par GitHub est vérifié puis échangé à chaque exécution.

Le pipeline de ce projet utilise la **clé JSON** (secret `GCP_SA_KEY`) : c'est le chemin le plus simple à mettre en place et à comprendre pour un premier déploiement, exactement ce dont vous avez besoin maintenant. Sa contrepartie est réelle : si ce secret fuit (dépôt mal configuré, log qui l'affiche par erreur), quiconque le récupère peut agir comme ce compte de service depuis n'importe où, jusqu'à ce que la clé soit révoquée manuellement.

À RETENIR Quand ce projet passera en usage réel (au-delà de l'exercice), basculez vers la Workload Identity Federation : aucun secret à faire fuiter, et l'accès peut être restreint à ce dépôt et cette branche précisément. Le fichier `ci-cd.yml` contient déjà, en commentaire, les deux lignes à activer pour ce basculement.

05 — ACTIVATION

Activer les services (API)

Chaque service GCP correspond à une API désactivée par défaut sur un nouveau projet : impossible de créer un service Cloud Run tant que `run.googleapis.com` n'a pas été explicitement activée. C'est une

protection double — contre les erreurs de manipulation et contre la facturation accidentelle d'un service jamais réellement utilisé.

Trois API suffisent pour ce projet, activées en une commande (déjà dans le `README`) :

`run.googleapis.com`

Cloud Run — exécution des conteneurs

`artifactregistry.googleapis.com`

Artifact Registry — stockage des images Docker

`secretmanager.googleapis.com`

Secret Manager — stockage des secrets applicatifs

06 — STOCKAGE DES IMAGES

Artifact Registry

Un registre Docker privé, hébergé par Google, découpé en **dépôts** (*repositories*) régionaux — ce projet en crée un seul, nommé `task-manager`, dans une région que vous choisissez (ex. `europa-west1`). Choisir la même région que Cloud Run réduit la latence et le coût du transfert au moment du déploiement.

Le nom complet d'une image suit toujours ce gabarit :

```
RÉGION-docker.pkg.dev/PROJECT_ID/DÉPÔT/IMAGE:TAG
```

... ce qui donne concrètement dans le pipeline : `europa-west1-docker.pkg.dev/task-manager-472013/task-manager/task-manager-backend:a1b2c3d4e5f6` .

La commande `gcloud auth configure-docker RÉGION-docker.pkg.dev` ne fait qu'une chose : elle écrit un assistant d'authentification dans `~/.docker/config.json`, pour qu'un simple `docker push` utilise en coulisses le jeton OAuth de votre session `gcloud` — sans elle, Docker ne sait pas comment prouver son identité au registre.

À RETENIR

Le tag `latest` est une étiquette *mobile* qui pointe simplement vers le dernier push — elle peut changer sous vos pieds. Le tag basé sur le SHA du commit est, lui, immuable. C'est pourquoi le job `deploy` déploie toujours l'image taguée par le SHA, jamais `:latest` : la révision Cloud Run en production correspond ainsi exactement à un commit identifiable.

07 — LE CŒUR DU SUJET

Cloud Run en détail

Cloud Run est une plateforme de conteneurs **serverless entièrement managée** : vous fournissez une image, Google s'occupe des machines, du système d'exploitation, des correctifs de sécurité et expose automatiquement une URL HTTPS publique (`https://SERVICE-HASH-REGION.a.run.app`). Aucun serveur à provisionner, aucun équilibreur de charge à configurer à la main.

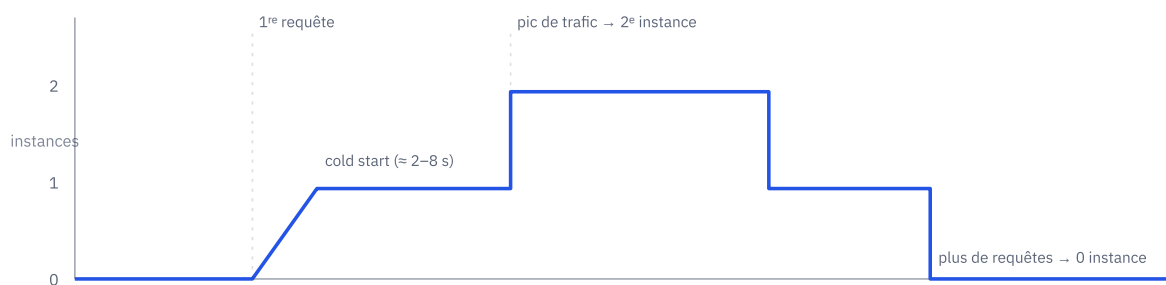
Le contrat avec votre conteneur tient en une phrase : **votre processus doit écouter sur le port indiqué par la variable d'environnement `$PORT`** que Cloud Run injecte (8080 par défaut) et répondre en HTTP dans le délai de démarrage configuré. C'est tout. C'est exactement pourquoi `server.port: ${PORT:8080}` dans `application.yaml` rendait déjà ce backend compatible Cloud Run, sans la moindre adaptation.

Révisions : chaque déploiement est immuable

Chaque appel de déploiement crée une nouvelle **révision** — un instantané figé (image + configuration). Une révision n'est jamais modifiée après coup ; « déployer » signifie créer une nouvelle révision puis lui router le trafic. Conséquence directe : revenir à la version précédente ne demande aucun redéploiement, juste re-router le trafic vers l'ancienne révision, encore présente. Le trafic peut même être partagé en pourcentage entre plusieurs révisions (déploiement canary) — pas nécessaire pour ce projet, mais bon à savoir que l'outil existe.

Mise à l'échelle et « scale-to-zero »

Cloud Run observe le nombre de requêtes simultanées et démarre ou arrête des instances de conteneur en conséquence, jusqu'à zéro par défaut : **vous ne payez rien tant que personne n'appelle l'API**. C'est le changement mental principal par rapport à un VPS classique : il n'y a pas de « serveur toujours allumé » à moins de fixer explicitly un nombre minimal d'instances.



Chaque palier correspond à une décision de l'autoscaler, pas à une action de votre part. Le seul levier que vous posez : `--min-instances` et `--max-instances`, qui bornent respectivement le plancher (0 = pas de coût au repos) et le plafond (protection contre un pic imprévu).

Concurrence, mémoire, et les indicateurs du pipeline

Chaque instance active peut traiter plusieurs requêtes en parallèle jusqu'à une limite de **concurrency** configurable (80 par défaut) — le Tomcat embarqué de Spring Boot est multi-thread, donc cela fonctionne nativement. Vous ne payez pas par requête, mais pour les secondes-instance réellement utilisées pendant qu'au moins une requête est en cours.

OPTION DU JOB <code>DEPLOY</code>	EFFET CONCRET
<code>--allow-unauthenticated</code>	Accorde <code>roles/run.invoker</code> à <code>allUsers</code> : l'API est publique, sans couche d'authentification Google. Correct ici puisque le frontend/mobile fait sa propre authentification JWT applicative.
<code>--port=8080</code>	Doit correspondre au port réellement écouté par le conteneur (voir <code>server.port</code>).

OPTION DU JOB DEPLOY

EFFET CONCRET

`--min-instances=0`

Scale-to-zero accepté : cold start occasionnel, zéro coût au repos.

`--max-instances=4`

Plafond dur — protège contre une facture qui s'emballe en cas de pic ou de bug (boucle infinie d'appels, par exemple).

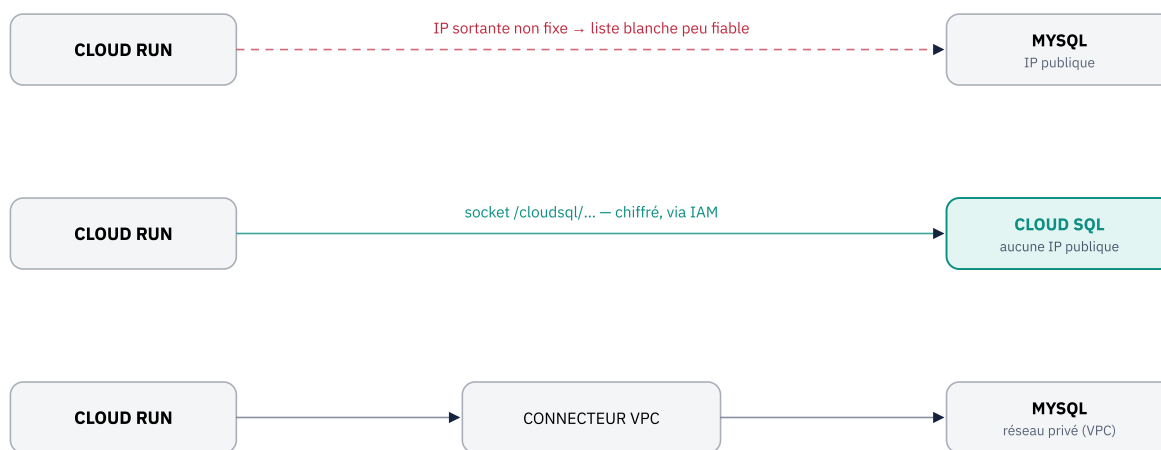
`--memory=512Mi`

Plafond mémoire du conteneur. Une JVM est gourmande ; 512 Mi est un point de départ raisonnable pour ce service, à augmenter en cas d'erreur *out of memory*.

08 — BASE DE DONNÉES

Joindre MySQL depuis Cloud Run

Cloud Run sait parler à Internet par défaut, mais ne vit pas à l'intérieur du réseau privé (VPC) de votre projet sauf configuration explicite. Trois façons de connecter le backend à MySQL, avec un compromis sécurité/simplicité différent à chaque fois.



Trois architectures pour le même besoin. Seul le chemin du milieu change — comparez-le, c'est là que se joue la décision.

PIÈGE N°2

« J'ai autorisé mon IP mais Cloud Run n'arrive toujours pas à se connecter. » Sans connecteur VPC, l'IP sortante d'un service Cloud Run **n'est pas fixe** — impossible de la mettre fiablement dans une liste blanche Cloud SQL. C'est précisément pour cette raison que Google a construit l'intégration native du **Cloud SQL Auth Proxy** (chemin du milieu ci-dessus) comme option recommandée pour connecter Cloud Run à Cloud SQL : on attache l'instance au déploiement (`--add-cloudsql-instances=PROJECT:RÉGION:INSTANCE`) et Cloud Run expose la base à un socket Unix local, sans jamais passer par une IP.

Pour ce projet, tant que MySQL tourne ailleurs qu'en Cloud SQL (VM, Docker Compose distant), le chemin 1 (IP publique + SSL forcé) reste le plus simple pour démarrer. Basculer vers Cloud SQL avec le socket `/cloudsql/...` est l'étape suivante logique si la base doit un jour devenir totalement privée — cela demande d'ajouter le connecteur `mysql-socket-factory-connector-j` côté Spring et d'ajuster l'URL JDBC.

09 — SECRETS APPLICATIFS

Secret Manager

Secret Manager est un coffre dédié aux chaînes sensibles (mots de passe, clés), séparé de la configuration normale : la *valeur* d'un secret ne transite jamais en clair dans la configuration Cloud Run, dans les logs CI, ni dans l'image Docker.

Chaque mise à jour d'un secret crée une nouvelle **version** ; l'ancienne reste accessible. On référence soit un numéro de version précis, soit l'alias `latest` — exactement ce que signifie `DB_PASSWORD:latest` dans le job `deploy`. L'accès en lecture est lui-même une permission IAM, accordée ici *secret par secret* (pas au niveau du projet) : la surface d'impact d'une fuite est réduite au strict minimum.

PIÈGE N°3

Deux identités distinctes entrent en jeu, pas une seule. Le compte de service qui *déploie* (celui de GitHub Actions) a besoin de `secretAccessor` pour *référencer* un secret au moment du déploiement. Mais le service Cloud Run *lui-même* s'exécute sous sa propre identité d'exécution — par défaut, le *compte de service Compute par défaut* du projet — et c'est **cette identité-là** qui doit, elle aussi, avoir `secretAccessor` pour pouvoir réellement lire la valeur au démarrage du conteneur. Oublier ce second octroi produit un service qui se déploie sans erreur... puis plante au démarrage avec une erreur de permission sur le secret. Le correctif : accorder `secretAccessor` également au compte de service d'exécution (ou en définir un dédié via `--service-account` au déploiement).

10 – COÛTS

Facturation

Un **compte de facturation** (moyen de paiement) doit être rattaché au projet avant de pouvoir activer la plupart des API — y compris celles qui resteront gratuites à l'usage. C'est une source fréquente de confusion pour un premier projet : « pourquoi dois-je entrer une carte pour un service gratuit ? » — parce que *gratuit* signifie ici « facturé zéro tant que le quota gratuit n'est pas dépassé », pas « sans compte de facturation ».

Un nouveau compte Google Cloud reçoit en général un crédit d'essai temporaire, distinct du **quota gratuit permanent** (*Always Free*) qui s'applique en continu, même après le crédit d'essai. Pour Cloud Run, ce quota permanent couvre chaque mois **2 millions de requêtes**, **360 000 Go-secondes** de mémoire et **180 000 vCPU-secondes** — largement suffisant pour un projet de démonstration comme celui-ci, qui restera probablement facturé **0 €/mois**.

coût

Filet de sécurité simple : dans *Facturation* → *Budgets et alertes*, créer un budget (ex. 5 €) qui déclenche un e-mail au franchissement — il ne coupe rien automatiquement, mais prévient tôt. Le vrai coupe-circuit de dépense reste `-max-instances` sur Cloud Run, déjà présent dans le pipeline : il borne le pire cas possible.

11 — SYNTHÈSE OPÉRATIONNELLE

Le pipeline, pas à pas

Ce qui se passe exactement, dans l'ordre, à chaque `git push` sur `main` — chaque étape porte le nom du job réel dans `.github/workflows/ci-cd.yml` :

- 1 **build** — compile le projet (`./gradlew assemble`), publie le `.jar` en artefact GitHub (traçabilité, pas utilisé par la suite).
- 2 **test** — exécute la suite de tests ; un test qui échoue arrête tout le pipeline ici, avant que quoi que ce soit ne soit poussé vers GCP.
- 3 **package** — authentification GCP (via `GCP_SA_KEY`), construction de l'image Docker (le `Dockerfile` multi-stage recompile le projet dans un conteneur), puis `docker push` de l'image taguée par le SHA du commit vers Artifact Registry.
- 4 **deploy** — appel de l'API Cloud Run Admin pour créer une nouvelle révision pointant vers cette image précise, avec les variables (`DB_HOST` , `DB_PORT` ...) et les références Secret Manager (`DB_PASSWORD` , `JWT_SECRET`) ; le trafic bascule à 100 % dessus dès qu'elle répond correctement au démarrage.

Sur une *pull request*, seuls `build` et `test` s'exécutent : validation avant fusion, aucun déploiement déclenché.

Se lancer, dans l'ordre

- 1 Créer un compte Google Cloud (console.cloud.google.com) et activer l'essai gratuit si proposé.
- 2 Créer un **projet** depuis la console (bouton « Nouveau projet ») et noter son **Project ID** généré — c'est la valeur qui ira dans la variable GitHub `GCP_PROJECT_ID`.
- 3 Lier un **compte de facturation** au projet (la console le propose automatiquement à la première tentative d'activation d'API).
- 4 Installer localement le **gcloud CLI**, puis `gcloud init` et `gcloud auth login`.
- 5 Exécuter dans l'ordre les commandes déjà présentes dans le `README.md` de ce projet (section *CI/CD*) : activation des API → création du dépôt Artifact Registry → création du compte de service → attribution des quatre rôles IAM → génération de la clé → création des secrets `DB_PASSWORD` et `JWT_SECRET`.
- 6 Renseigner les secrets et variables GitHub listés dans le même README (`GCP_SA_KEY` , `GCP_PROJECT_ID` , `GCP_REGION` , `DB_HOST` ...).
- 7 Pousser sur `main` , suivre l'exécution dans l'onglet **Actions** du dépôt, puis dans la console GCP sous **Cloud Run** → **task-manager-backend** → **Révisions**.
- 8 Une fois déployé, tester l'URL publique affichée en sortie du job `deploy` : `curl https://...a.run.app/api/auth/login` doit répondre (même une erreur 400 de validation prouve que le service tourne).

Lexique & pièges courants

Projet

Unité d'isolation racine de GCP — facturation, permissions et quotas y sont tous rattachés.

Compte de service

Identité non humaine (« robot ») utilisée par une application ou un pipeline pour appeler les API GCP.

Rôle IAM

Paquet nommé de permissions, accordé à un principal sur une ressource.

Révision

Instantané immuable d'un déploiement Cloud Run — chaque déploiement en crée une nouvelle.

Cold start

Délai de démarrage d'une nouvelle instance quand aucune n'était disponible pour répondre.

Artifact Registry

Registre Docker privé et régional de GCP.

Secret Manager

Coffre-fort versionné pour les valeurs sensibles, consommé via IAM.

Concurrency

Nombre maximal de requêtes traitées en parallèle par une même instance Cloud Run.

VPC

Réseau privé virtuel d'un projet — Cloud Run n'y est pas connecté par défaut.

Symptôme → cause probable

SYMPTÔME	CAUSE LA PLUS PROBABLE
« Permission denied to act as service account »	Le rôle <code>iam.serviceAccountUser</code> manque sur le compte de service cible (section 03).
« Image not found » au déploiement	Région erronée dans le chemin de l'image, ou <code>gcloud auth configure-docker</code> jamais exécuté (section 06).
Le service plante au démarrage sur Cloud Run mais tourne en local	Port codé en dur au lieu de lire <code>\$PORT</code> , ou compte de service d'exécution sans <code>secretAccessor</code> (section 09).
« J'ai autorisé mon IP mais la connexion à la base échoue »	L'IP sortante de Cloud Run n'est pas fixe sans connecteur VPC (section 08).
Facture plus élevée que prévu	<code>--max-instances</code> absent, ou <code>--min-instances</code> réglé au-dessus de 0 par mégarde (section 10).

Ce document décrit le fonctionnement de GCP tel qu'appliqué au pipeline `.github/workflows/ci-cd.yml` et au `README.md` de `recutement-test` — pas un cours général sur le cloud. Les commandes de mise en place restent la source de vérité dans le `README` ; ce guide en explique le pourquoi.